# Principles of Pipelining

# Basic Computer Architecture

# Outline

mop

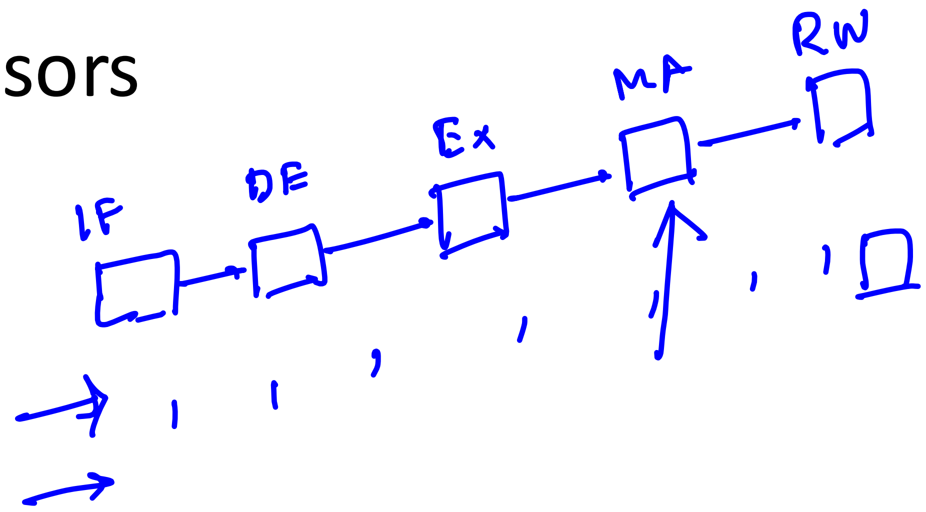
- Overview of Pipelining
- A Pipelined Data Path
- Pipeline Hazards
- Pipeline with Interlocks
- Forwarding
- Performance Metrics
- Interrupts/ Exceptions

# Up till now ….

* We have designed a processor that can execute all the SimpleRisc Instructions
* We have look at two styles :
  * With a hardwired control unit
  * Microprogrammed control unit
    * Microprogrammed data path
    * Microassembly Language
    * Microinstructions

# Designing Efficient Processors

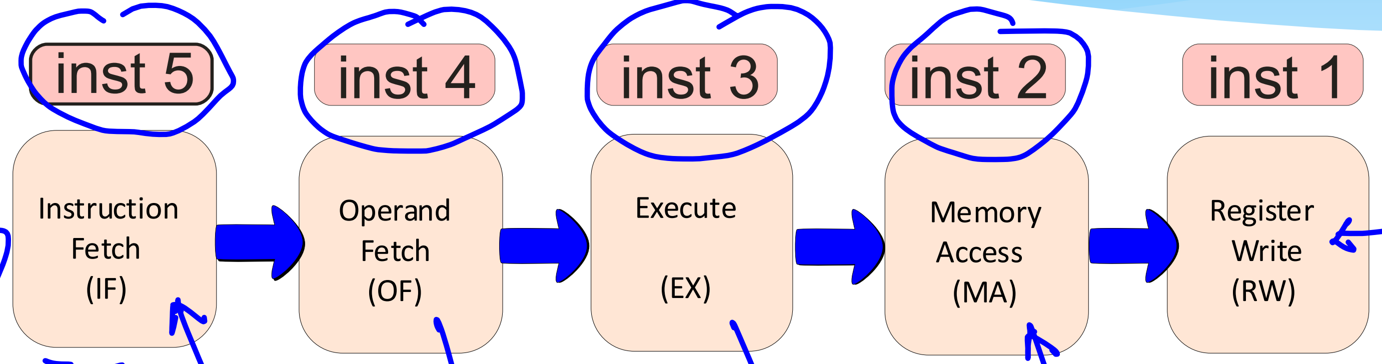
* Microprogrammed processors are much slower that hardwired processors
* Even hardwired processors
  * Have a lot of waste !!!
  * We have 5 stages.
  * What is the IF stage doing, when the MA stage is active ?
    * **ANSWER** : It is **idling**



# The Notion of Pipelining

- Let us go back to the car assembly line
  - Is the engine shop idle, when the paint shop is painting a car ?
  - **NO** : It is building the engine of another car
  - When this engine goes to the body shop, it builds the engine of another car, and so on ….
  - **Insight** :
    - Multiple cars are built at the same time.
    - A car proceeds from one stage to the next

# Pipelined Processors

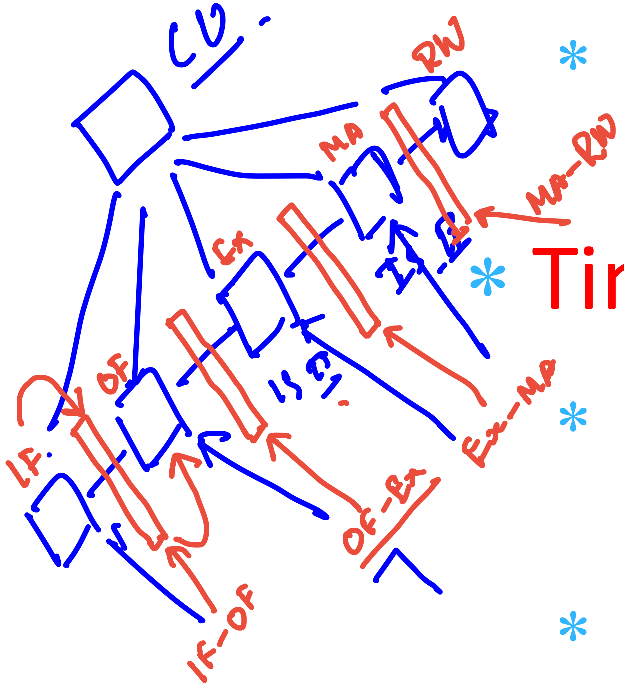



* The IF, ID, EX, MA, and RW stages process 5 instructions simultaneously
* Each instruction proceeds from one stage to the next
* This is known as pipelining

# Advantages of Pipelining

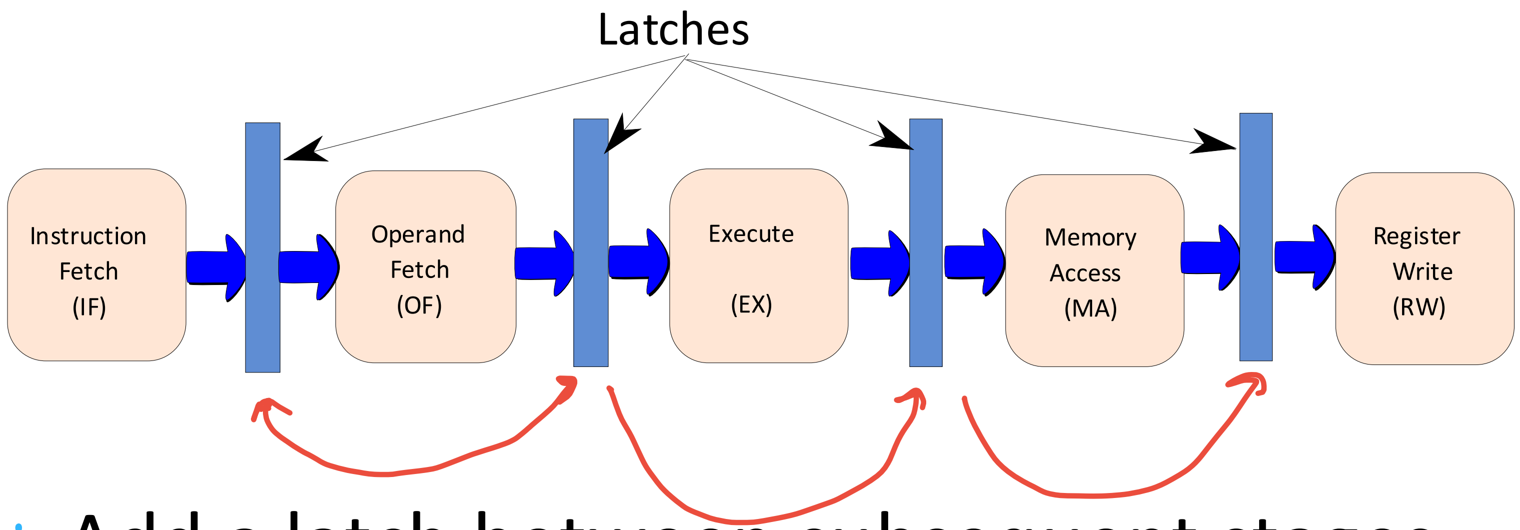
* We keep all parts of the data path, busy all the time
* Let us assume that all the 5 stages do the same amount of **work**
  * Without pipelining, every T seconds, an instruction completes its execution
  * With pipelining, every T/5 seconds, a new instruction completes its execution

# Design of a Pipeline

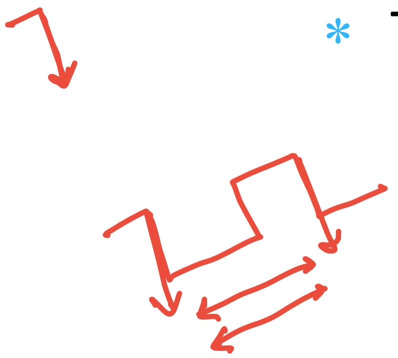



* Splitting the Data Path
  * We divide the data path into 5 parts : IF, OF, EX, MA, and RW
* Timing
  * We insert latches (pipeline registers) between consecutive stages
  * 4 Latches → IF-OF, OF-EX, EX-MA, and MA-RW
  * At the negative edge of a clock, an instruction moves from one stage to the next

# Pipelined Data Path with Latches




* Add a latch between subsequent stages.
  * Triggered by a negative clock edge



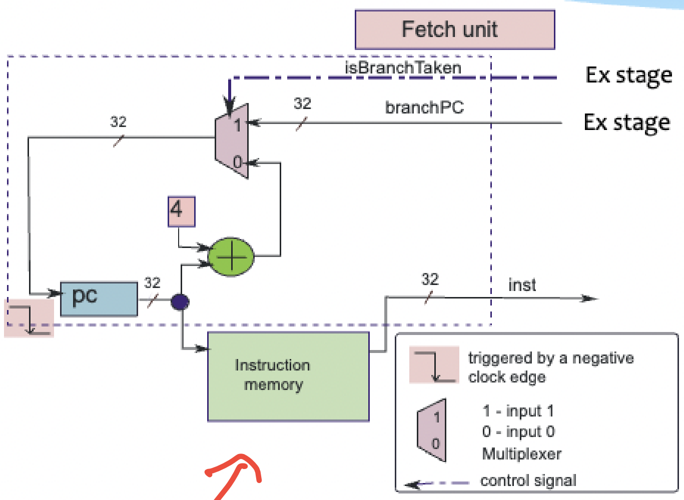
# The Instruction Packet

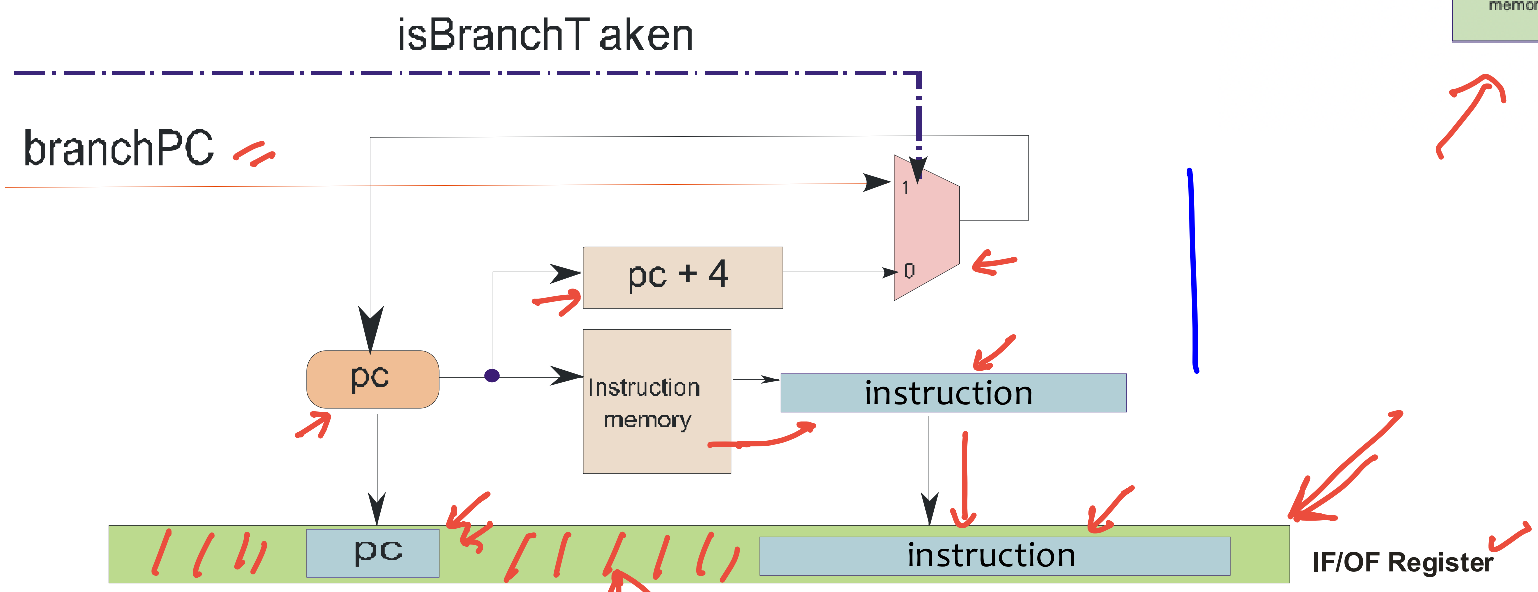
- What travels between stages ?
  - **ANSWER** : the instruction packet
- Instruction Packet
  - Instruction contents
  - Program counter
  - All intermediate results
  - Control signals
- Every instruction moves with its entire state, no interference between instructions

# Outline

* Overview of Pipelining
* A Pipelined Data Path
* Pipeline Hazards
* Pipeline with Interlocks
* Forwarding
* Performance Metrics
* Interrupts/ Exceptions

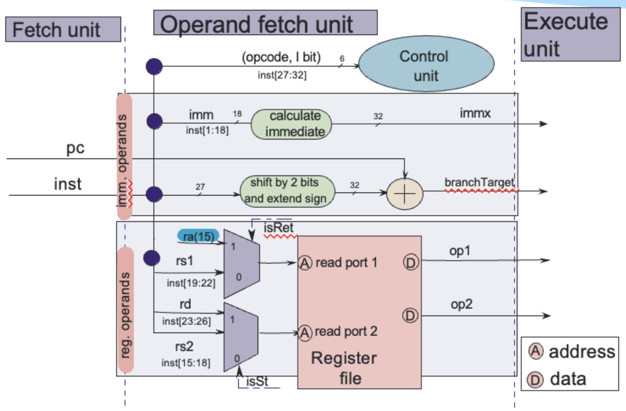
# IF Stage

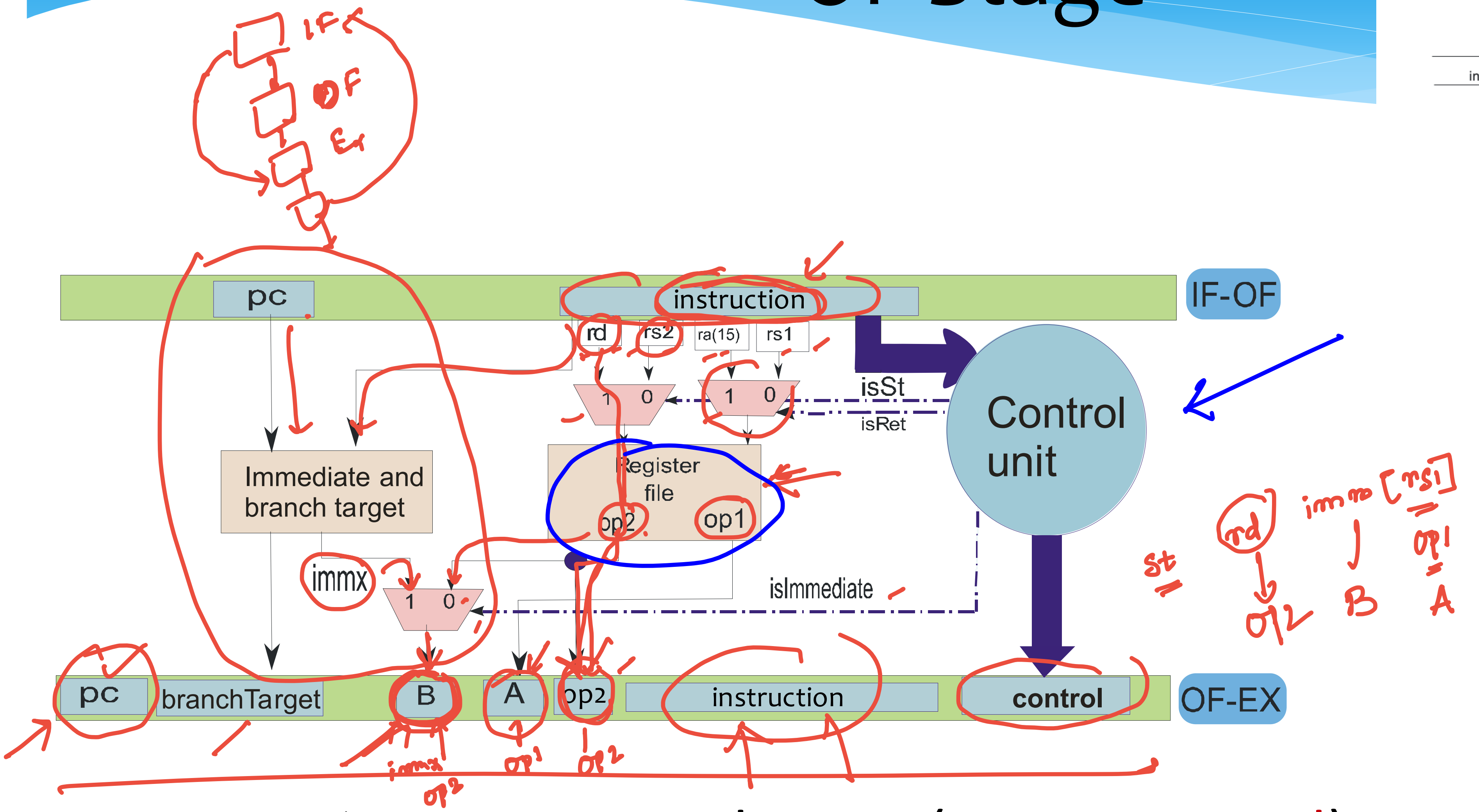



* Instruction contents saved in the instruction field

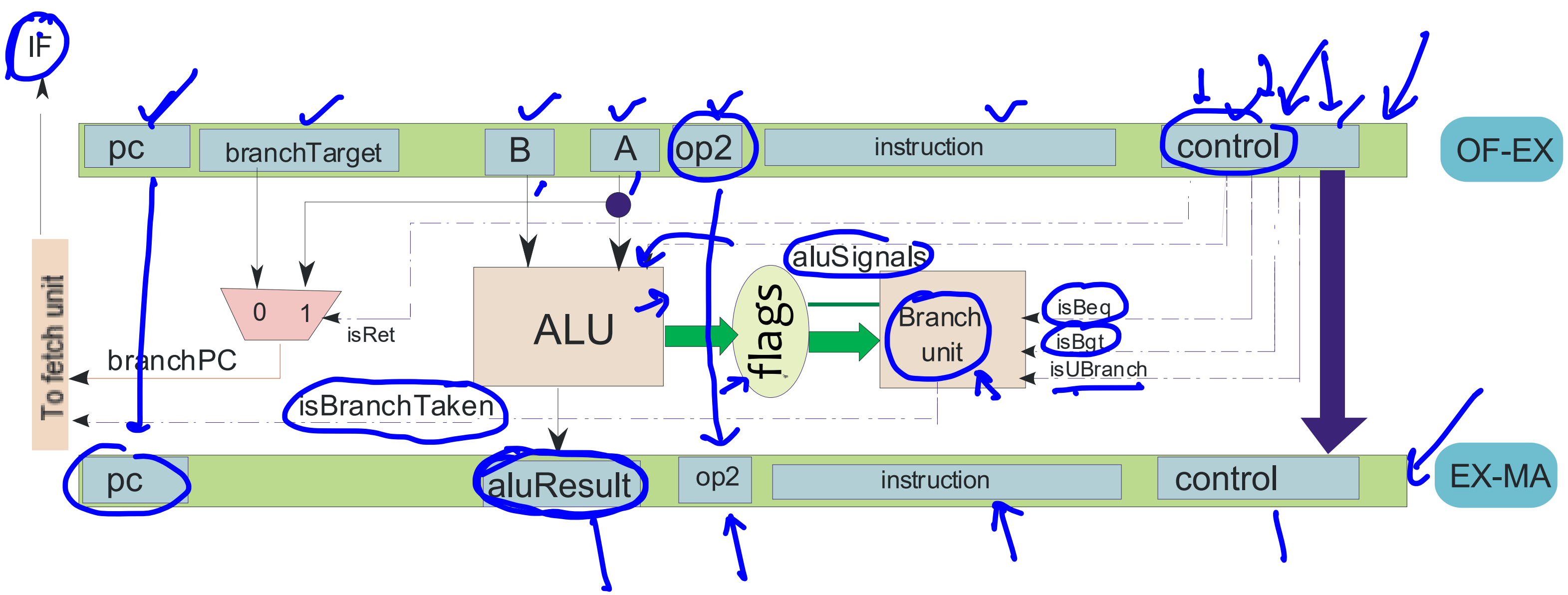
# OF Stage

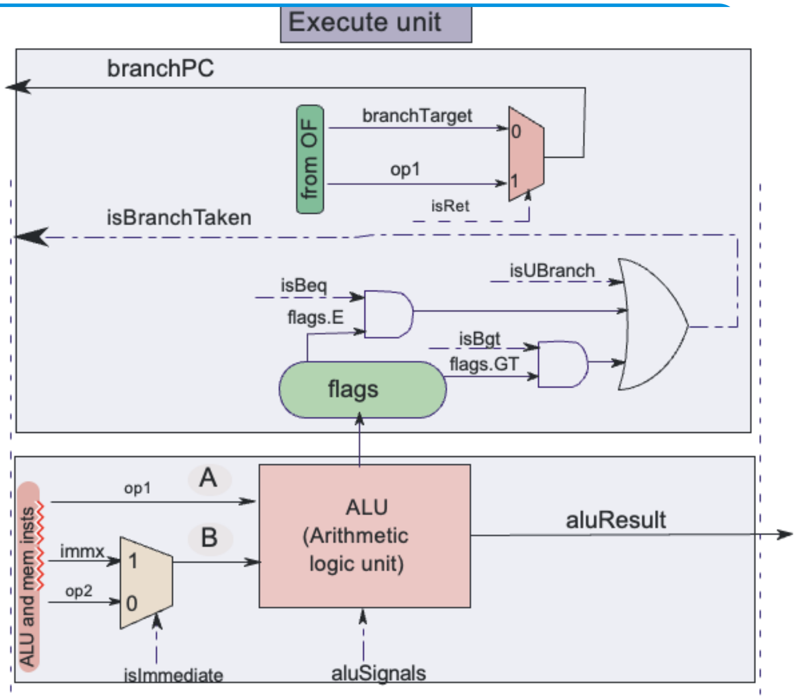



* A, B → ALU Operands, op2 (store operand), control (set of all control signals)

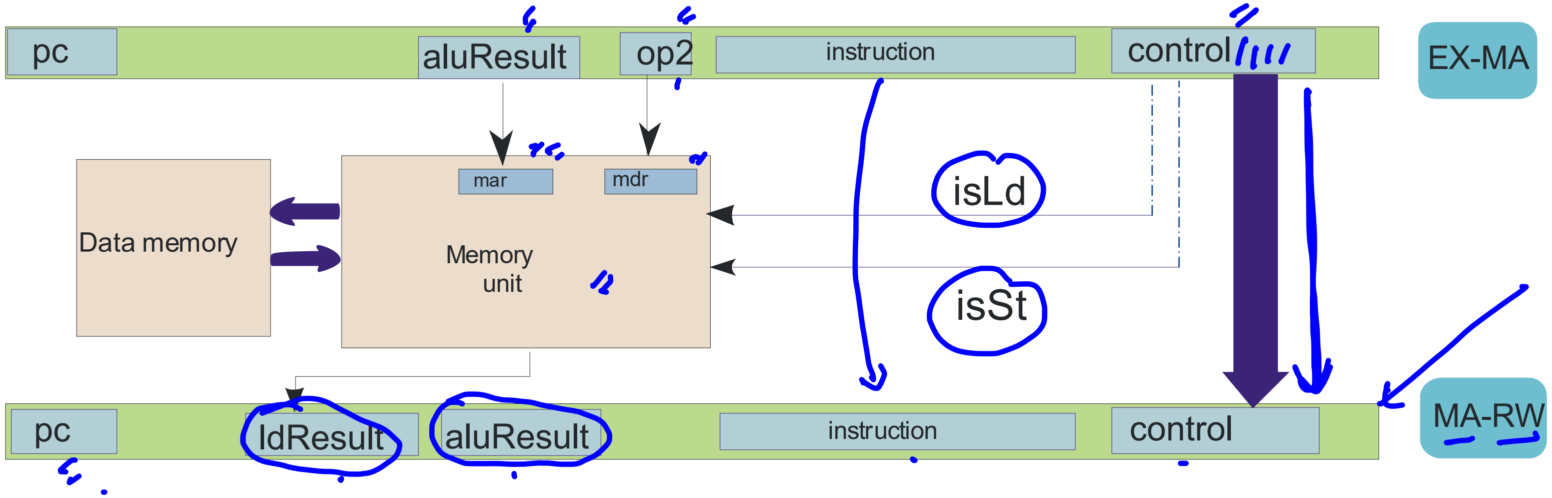
# EX Stage

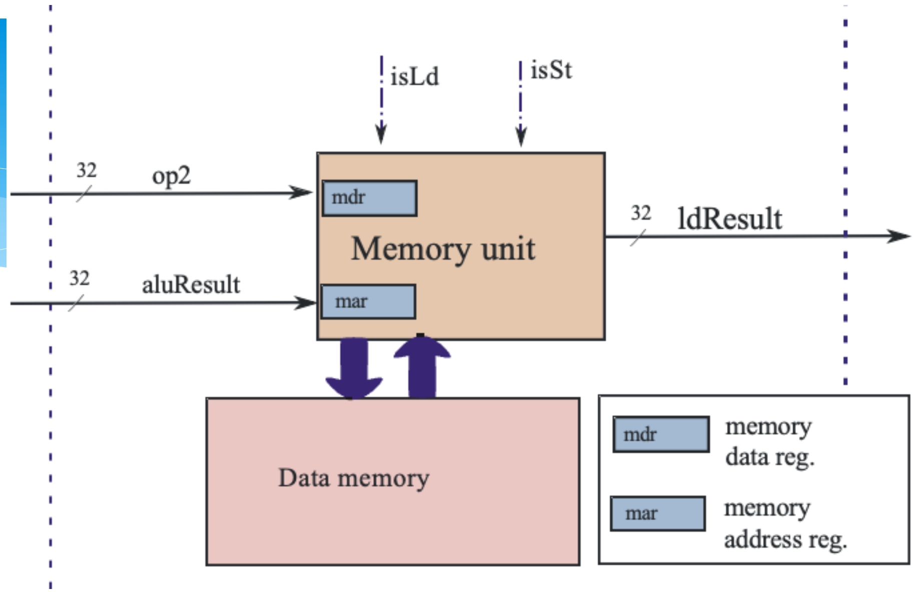



* aluResult → result of the ALU Operation
* op2, control, pc, instruction (passed from OF-EX)

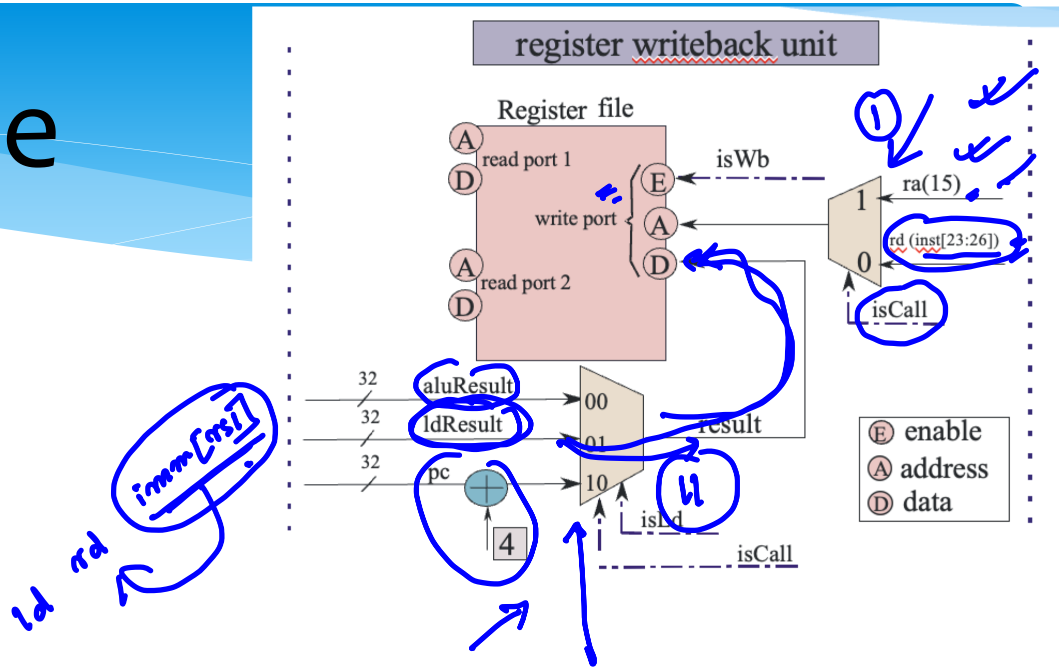
# MA Stage



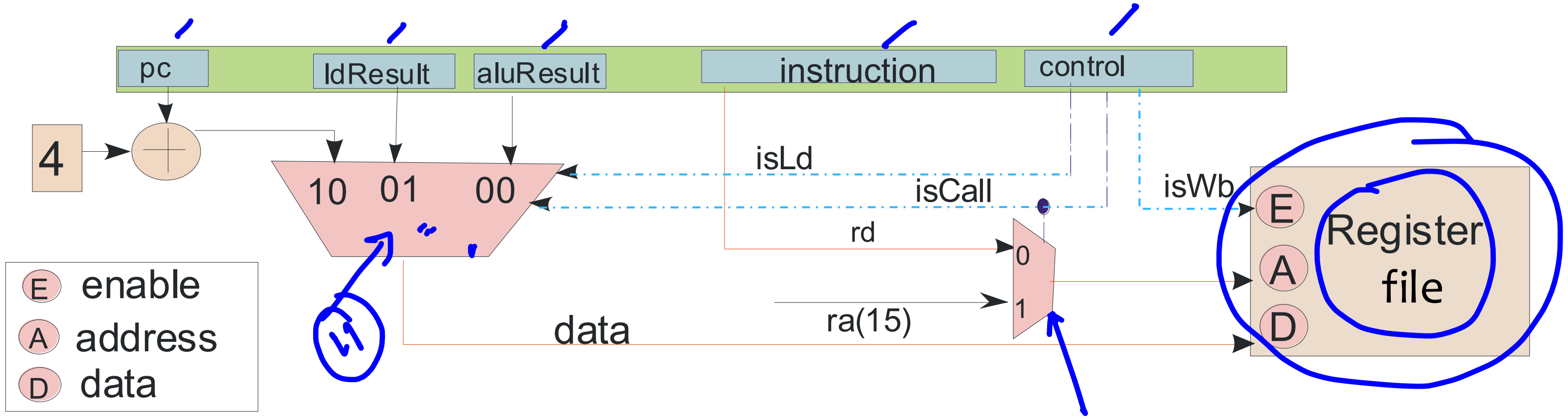




* ldResult → result of the load operation
* aluResult, control, pc, instruction (passed from EX-MA)

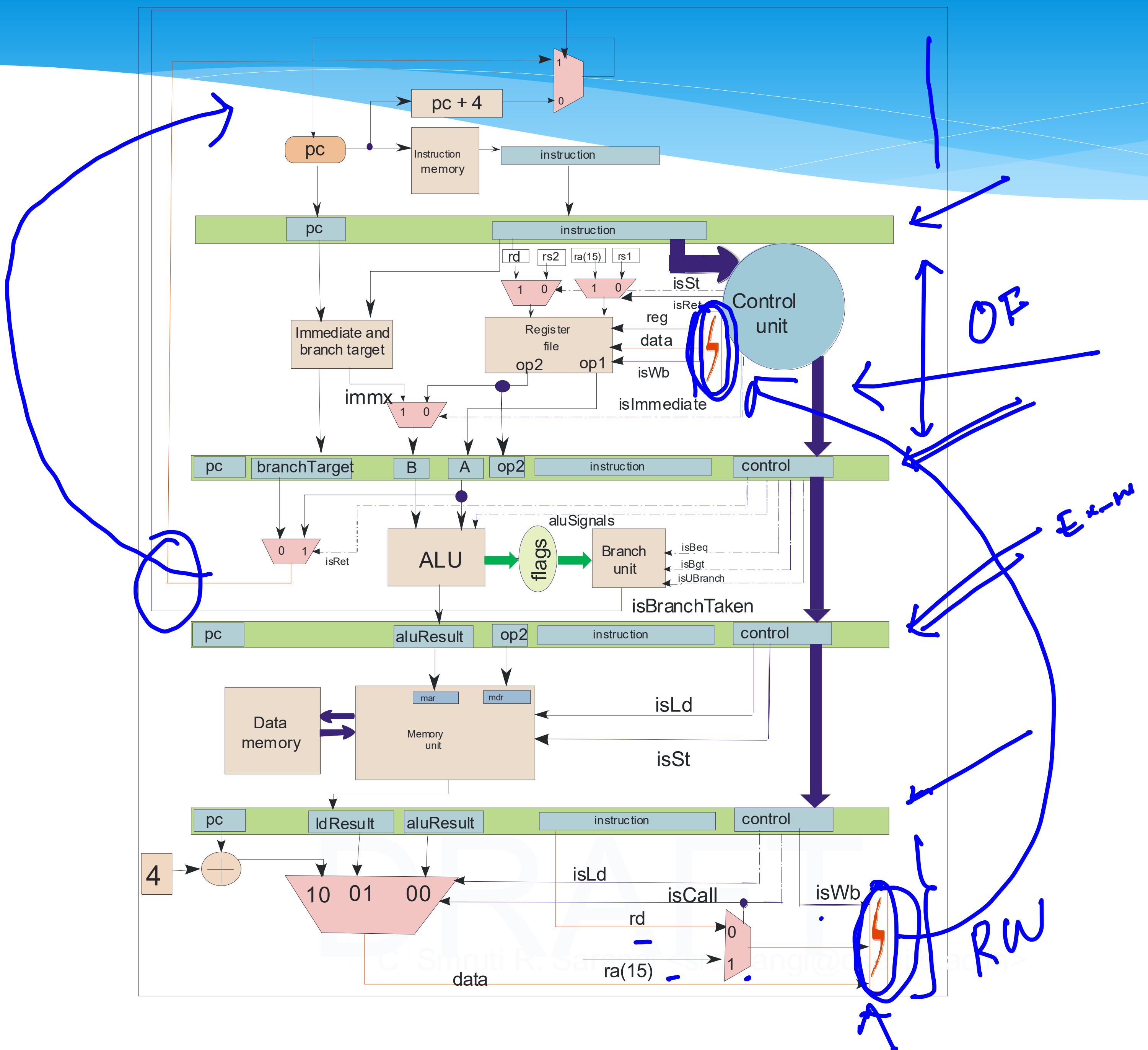
# RW Stage



register writeback unit
Register file
read port 1
write port
read port 2
isWb
ra(15)
rd (inst[23:26])
isCall
aluResult
ldResult
pc
result
4
isLd
isCall
E enable
A address
D data
imm[rs1]
ld
rd

pc
ldResult
aluResult
instruction
control
4
10
01
00
isLd
isCall
isWb
rd
ra(15)
data
Register file
E
A
D
E enable
A address
D data


pc + 4
pc
Instruction memory
instruction
pc
instruction
rd
rs2
ra(15)
rs1
isSt
isRet
Control unit
reg
data
isWb
Register file
op2
op1
Immediate and branch target
immx
isImmediate
OF
pc
branchTarget
B
A
op2
instruction
control
aluSignals
isRet
ALU
flags
Branch unit
isBeq
isBgt
isUBranch
isBranchTaken
Ex-M
pc
aluResult
op2
instruction
control
mar
mdr
Data memory
Memory unit
isLd
isSt
pc
ldResult
aluResult
instruction
control
4
10
01
00
isLd
isCall
isWb
rd
ra(15)
data
RW